

BIKE SHARING DEMAND FORECASTING

*A Comparative Study of Statistical and Machine Learning
Approaches*

Naga Venkata Sai Ram Charan Pullabhotla

Student ID: 24071651

GitHub Link: https://github.com/sairampullabhotla1992/Bike_rental_project.git

M.Sc. Data Science

University of Hertfordshire

April 2026

Table of Contents

1. Abstract
2. Introduction
 - 2.1 Background and Motivation
 - 2.2 Research Question and Objectives
 - 2.3 Report Structure
3. Literature Review
 - 3.1 Traditional Time-Series Methods
 - 3.2 Machine Learning Approaches
 - 3.3 Ensemble and Hybrid Methods
 - 3.4 Gaps in Existing Literature
4. Methodology
 - 4.1 Dataset Description
 - 4.2 Exploratory Data Analysis
 - 4.3 Feature Engineering
 - 4.4 Train-Test Split and Target Transformation
 - 4.5 Model Specifications
 - 4.6 Hyperparameter Tuning Strategy
 - 4.7 Evaluation Metrics
5. Results and Analysis
 - 5.1 SARIMA Results
 - 5.2 SARIMAX Results
 - 5.3 XGBoost Results
 - 5.4 LightGBM Results
 - 5.5 Multi-Horizon Forecasting
 - 5.6 Ensemble Models
 - 5.7 Comprehensive Comparison
 - 5.8 Feature Importance Analysis
 - 5.9 Residual Analysis
6. Discussion
 - 6.1 Key Findings
 - 6.2 Comparison with Literature
 - 6.3 Limitations
 - 6.4 Ethical Considerations
7. Conclusion and Future Work
8. References

1. Abstract

This report presents a comprehensive comparative study of statistical and machine learning approaches for short-term hourly bike rental demand forecasting using the UCI Bike Sharing Dataset (Fanaee-T and Gama, 2014). The study addresses a key operational challenge in urban mobility: predicting hourly bike demand to enable efficient fleet rebalancing and resource allocation. We implement and systematically compare four model families — SARIMA, SARIMAX, XGBoost, and LightGBM — along with three weighted ensemble strategies, evaluating all models using MAE, RMSE, and WMAPE (Weighted Mean Absolute Percentage Error).

Key methodological contributions include: (1) log1p transformation for SARIMA/SARIMAX to handle multiplicative demand patterns, (2) rolling Kalman filter one-step-ahead forecasting for realistic evaluation (avoiding the pitfalls of one-shot forecasting), (3) engineering of 42 domain-specific features incorporating cyclical encoding, trend, weather interactions, lag features, and rolling statistics, (4) proper hyperparameter tuning via RandomizedSearchCV with TimeSeriesSplit cross-validation, and (5) multi-horizon forecasting analysis at 1, 6, and 24 hours ahead.

The best-performing model — a triple weighted ensemble of SARIMAX, LightGBM, and XGBoost — achieves a WMAPE of 12.64%, demonstrating that combining statistical time-series models with gradient boosting methods yields superior forecasting accuracy. Machine learning models halve the forecasting error compared to statistical approaches (WMAPE ~13% vs ~23%), primarily through their ability to capture non-linear demand patterns and complex feature interactions.

2. Introduction

2.1 Background and Motivation

Urban mobility is increasingly shifting towards sustainable transportation options, with bike-sharing systems growing rapidly in cities worldwide. According to the National Association of City Transportation Officials, bike-sharing trips in major cities increased significantly between 2010 and 2020, driven by environmental awareness, health benefits, and last-mile connectivity needs. For bike-sharing operators, the ability to predict demand accurately at an hourly granularity is critical for several operational decisions: fleet rebalancing between stations, maintenance scheduling, staffing allocation, and capacity planning for new stations.

Washington D.C.'s Capital Bikeshare system, one of the largest in the United States, serves as an ideal case study. The system exhibits complex demand patterns driven by commuter schedules (morning and evening rush hours), weather conditions (temperature, humidity, wind), and calendar effects (weekday vs weekend, holidays, seasons). These multi-factor dependencies make demand forecasting a challenging multivariate time-series regression problem that requires both temporal modelling and the incorporation of exogenous explanatory variables.

2.2 Research Question and Objectives

This project addresses the primary research question: Can historical bike usage data and weather information accurately forecast short-term (hourly) bike rental demand using time-series and machine learning models? Additionally, we investigate the following sub-questions:

- Do exogenous weather and calendar features improve statistical model accuracy (SARIMA vs SARIMAX)?
- Can gradient boosting methods (XGBoost, LightGBM) significantly outperform traditional time-series models?
- Does ensembling statistical and ML models provide additional forecasting improvements?
- How does forecast accuracy degrade across different prediction horizons (1h, 6h, 24h)?

2.3 Report Structure

Section 3 reviews relevant literature on demand forecasting methods. Section 4 details the methodology including dataset description, feature engineering, model specifications, and evaluation metrics. Section 5 presents results across all models and ensemble strategies. Section 6 discusses findings, limitations, and

ethical considerations. Section 7 concludes with key contributions and directions for future work.

3. Literature Review

3.1 Traditional Time-Series Methods

The SARIMA (Seasonal AutoRegressive Integrated Moving Average) family of models has been the cornerstone of time-series forecasting since the seminal work of Box and Jenkins (1970). Hyndman and Athanasopoulos (2021) provide a comprehensive modern treatment of SARIMA methodology, including ACF/PACF-based order selection, seasonal differencing, and the critical importance of rolling (walk-forward) evaluation for honest forecast assessment. The SARIMAX extension incorporates exogenous regressors, allowing the model to leverage weather and calendar information alongside the autoregressive temporal structure.

Chen et al. (2020) specifically applied hybrid SARIMA models with data-driven feature engineering for short-term bike-sharing demand prediction, publishing in Transportation Research Record. They demonstrated that cyclical encoding of temporal features (using sine/cosine transformations) improves the performance of linear time-series models by preserving the circular continuity of periodic variables — a finding we incorporate directly into our feature engineering pipeline.

3.2 Machine Learning Approaches

Chen and Guestrin (2016) introduced XGBoost, a scalable gradient boosting framework that handles non-linear relationships and feature interactions natively through its tree-based architecture. The regularised objective function combines a training loss with complexity penalty terms (L1 and L2 regularisation), providing protection against overfitting. XGBoost uses level-wise tree growth and employs a second-order Taylor expansion of the loss function for efficient split finding.

Ke et al. (2017) proposed LightGBM, which differs from XGBoost in its leaf-wise tree growth strategy with maximum depth constraints. This approach grows trees by splitting the leaf with the highest loss reduction, leading to asymmetric trees that can model complex patterns with fewer splits. LightGBM also introduces Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB) for faster training on large datasets. Both methods have been successfully applied to demand forecasting across transportation, energy, and retail domains.

3.3 Ensemble and Hybrid Methods

Ensemble methods combine predictions from multiple base models to reduce variance and improve robustness. Weighted averaging is a simple but effective strategy where the contribution of each model is proportional to its predictive accuracy. In the context of demand forecasting, combining statistical models (which excel at capturing autoregressive patterns) with machine learning models (which handle non-linear feature interactions) can exploit complementary strengths. Li et al. (2019) demonstrated the value of multi-view learning approaches for traffic-related time series, supporting the idea that diverse model perspectives improve overall forecasting accuracy.

3.4 Gaps in Existing Literature

Our work addresses several gaps: (1) Most prior studies compare only 2–3 models; we provide a systematic comparison across four model families plus three ensemble strategies on the same dataset with consistent evaluation. (2) Many studies use MAPE, which is problematic for near-zero demand hours; we adopt WMAPE for robust evaluation. (3) Few studies examine multi-horizon forecast degradation, which is critical for operational planning. (4) Rolling Kalman filter evaluation of SARIMA/SARIMAX models is rarely implemented, despite being more realistic than one-shot forecasting.

4. Methodology

4.1 Dataset Description

The UCI Bike Sharing Dataset (Fanaee-T and Gama, 2014) contains 17,379 hourly records from the Capital Bikeshare system in Washington D.C., spanning January 2011 to December 2012. The target variable is cnt (total hourly bike rentals combining registered and casual users), ranging from 1 to 977 with a mean of 189.5 and standard deviation of 181.4. Key predictor variables include normalised temperature (temp, 0–1 scale), normalised humidity (hum), normalised wind speed (windspeed), weather situation (weathersit, categorical 1–4), and calendar information (hour, weekday, month, holiday, working day).

The dataset is loaded and indexed chronologically using a derived timestamp field. No missing values are present, and the data requires no imputation. The following code demonstrates the initial data loading and timestamp construction:

```
df = pd.read_csv("hour.csv")
df["dteday"] = pd.to_datetime(df["dteday"])
df["timestamp"] = df["dteday"] + pd.to_timedelta(df["hr"], unit="h")
df = df.sort_values("timestamp").set_index("timestamp")
```

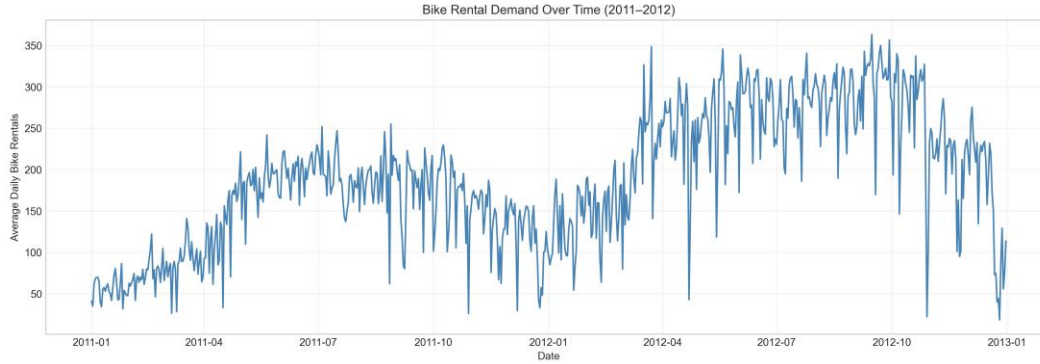


Figure 1: Bike Rental Demand Over Time (2011–2012)

4.2 Exploratory Data Analysis

The time-series plot (Figure 1) reveals a clear upward trend over the two-year period, reflecting growth in the bike-sharing system's user base. Strong daily (24-hour) seasonality is visible, with pronounced morning (7–9 AM) and evening (5–7 PM) commuter peaks on weekdays (Figure 2). Weekend demand is flatter and shifted towards midday leisure rides. Monthly patterns show higher demand in warmer months (May–October) and significantly reduced usage in winter.

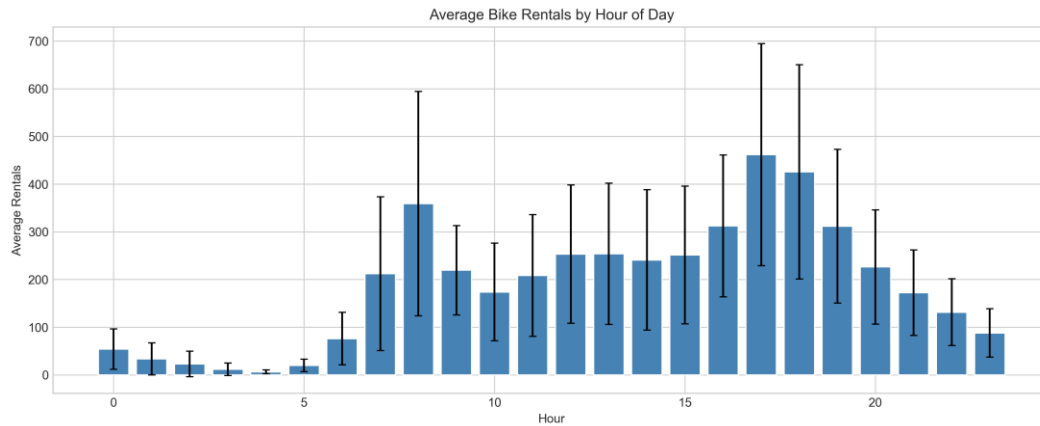


Figure 2: Average Bike Rentals by Hour of Day

The correlation heatmap (Figure 3) reveals key relationships: temperature has a moderate positive correlation with demand ($r = 0.39$), humidity shows a weak negative correlation ($r = -0.32$), and temperature and apparent temperature (atemp) are nearly perfectly correlated ($r = 0.99$), indicating multicollinearity. We retain temp and exclude atemp from the model features to avoid redundancy.



Figure 3: Correlation Heatmap — Before Feature Engineering

4.3 Feature Engineering

We engineer 42 features organised into six categories to capture different aspects of the demand signal. This section describes each category with the mathematical formulations and Python code used in the implementation.

Cyclical Encoding (8 features)

Temporal variables such as hour, weekday, and month are inherently circular: hour 23 is one hour away from hour 0, not 23 hours. We apply sine/cosine transformations to preserve this circular continuity:

$$hr_sin = \sin(2\pi \times hr / 24), \quad hr_cos = \cos(2\pi \times hr / 24) \quad (\text{Eq. 1})$$

$$weekday_sin = \sin(2\pi \times weekday / 7), \quad weekday_cos = \cos(2\pi \times weekday / 7) \quad (\text{Eq. 2})$$

$$month_sin = \sin(2\pi \times month / 12), \quad month_cos = \cos(2\pi \times month / 12) \quad (\text{Eq. 3})$$

$$dayofyear_sin = \sin(2\pi \times dayofyear / 365), \quad dayofyear_cos = \cos(2\pi \times dayofyear / 365) \quad (\text{Eq. 4})$$

```
df["hr_sin"] = np.sin(2 * np.pi * df["hr"] / 24)
df["hr_cos"] = np.cos(2 * np.pi * df["hr"] / 24)
```

```
df["weekday_sin"] = np.sin(2 * np.pi * df["weekday"] / 7)
df["weekday_cos"] = np.cos(2 * np.pi * df["weekday"] / 7)
```

Trend Feature (1 feature)

A linear trend variable captures the year-over-year growth in bike-sharing ridership:

$$trend = [0, 1, 2, \dots, N-1] \quad \text{where } N = 17,379 \quad (\text{Eq. 5})$$

```
df["trend"] = np.arange(len(df))
```

Binary Flags (3 features)

Domain-informed binary indicators capture operationally important demand regimes:

```
df["rush_hour"] = df["hr"].isin([7, 8, 9, 17, 18, 19]).astype(int)
df["is_weekend"] = df["weekday"].isin([0, 6]).astype(int)
df["peak_season"] = df["mnth"].isin([5, 6, 7, 8, 9, 10]).astype(int)
```

Interaction Features (6 features)

Interaction terms capture non-additive effects between variables. For example, the effect of temperature on demand depends on the time of day (cycling in warm weather is more attractive during commuting hours):

$$temp_sq = temp^2, \quad temp_hum = temp \times humidity \quad (\text{Eq. 6})$$

$$temp_x_hr = temp \times hr_sin, \quad temp_x_rush = temp \times rush_hour \quad (\text{Eq. 7})$$

```
df["temp_sq"] = df["temp"] ** 2
df["temp_hum"] = df["temp"] * df["hum"]
df["temp_x_hr"] = df["temp"] * df["hr_sin"]
df["temp_x_rush"] = df["temp"] * df["rush_hour"]
```

Lag Features (8 features)

Lag features provide autoregressive memory, allowing the model to use recent demand history as predictive input. We include lags at 1, 2, 3, 6, 12, 24, 48, and 168 hours (one week). Critically, all lags use shift(lag) to ensure only past data is used, preventing data leakage:

$$lag_kh = cnt(t - k) \quad \text{for } k \in \{1, 2, 3, 6, 12, 24, 48, 168\} \quad (\text{Eq. 8})$$

```
for lag in [1, 2, 3, 6, 12, 24, 48, 168]:
    df[f"lag_{lag}h"] = df["cnt"].shift(lag)
```

Rolling Statistics (8 features)

Rolling window statistics smooth out noise and provide contextual demand information. We apply shift(1) before computing rolling statistics to ensure no future information leaks into the features:

$$roll_wh = mean(cnt(t-w-1), \dots, cnt(t-1)) \quad \text{for } w \in \{3, 6, 12, 24, 168\} \quad (\text{Eq. 9})$$

```
for w in [3, 6, 12, 24, 168]:
    df[f"roll_{w}h"] = df["cnt"].shift(1).rolling(w).mean()
df[f"roll_24h_std"] = df["cnt"].shift(1).rolling(24).std()
df[f"roll_24h_min"] = df["cnt"].shift(1).rolling(24).min()
df[f"roll_24h_max"] = df["cnt"].shift(1).rolling(24).max()
```

Category	Features	Count
----------	----------	-------

Cyclical encoding	hr, weekday, month, dayofyear (sin/cos)	8
Trend	Linear trend (0 to N)	1
Binary flags	rush_hour, is_weekend, peak_season	3
Polynomial/Interaction	temp_sq, temp_hum, temp_x_hr, hum_x_hr, wind_x_hr, temp_x_rush	6
Lag features	1, 2, 3, 6, 12, 24, 48, 168 hours	8
Rolling statistics	Mean (3,6,12,24,168h), 24h std/min/max	8
SARIMAX exogenous	Weather + calendar + cyclical + engineered	17

Table 1: Summary of 42 Engineered Features

4.4 Train-Test Split and Target Transformation

The data is split 90/10 chronologically: 15,641 hours for training and 1,738 hours for testing. No random shuffling is applied to preserve temporal order — this is essential for time-series problems where future data must never leak into the training set. For SARIMA and SARIMAX models, the target is transformed using the log1p function to stabilise the multiplicative variance inherent in hourly demand data:

$$y_{\text{transformed}} = \log(1 + \text{cnt}) \quad (\text{Eq. 10})$$

$$y_{\text{predicted}} = \exp(\hat{y}) - 1 \quad (\text{inverse transform for evaluation}) \quad (\text{Eq. 11})$$

```
split = int(len(y) * 0.9) # 15,641 train / 1,738 test
y_train, y_test = y.iloc[:split], y.iloc[split:]
y_log = np.log1p(y)
y_train_log, y_test_log = y_log.iloc[:split], y_log.iloc[split:]
```

ML models (XGBoost, LightGBM) operate on raw cnt values, as tree-based methods handle skewed distributions natively without requiring variance-stabilising transformations.

4.5 Model Specifications

SARIMA(1,1,1)(1,1,1,24)

The SARIMA order is determined via ACF/PACF analysis of the doubly-differenced (trend + seasonal) series. After applying first-order differencing (d=1) and seasonal differencing at lag 24 (D=1), the ACF shows a cutoff after lag 1, suggesting MA(1), and the PACF shows a significant spike at lag 1, suggesting AR(1). Seasonal spikes at lag 24 in both ACF and PACF support seasonal orders of P=1 and Q=1. The model is formulated as:

$$\Phi(B^{24}) \varphi(B) \nabla_{24} \nabla_1 y_t = \Theta(B^{24}) \theta(B) \varepsilon_t \quad (\text{Eq. 12})$$

where B is the backshift operator, $\nabla_1 = (1-B)$, $\nabla_{24} = (1-B^{24})$, and φ , θ , Φ , Θ are the AR, MA, seasonal AR, and seasonal MA polynomials respectively. The model is fitted on the log1p-transformed training data using maximum likelihood estimation with memory conservation flags (set_conserve_memory) to manage the 51-dimensional state space efficiently.

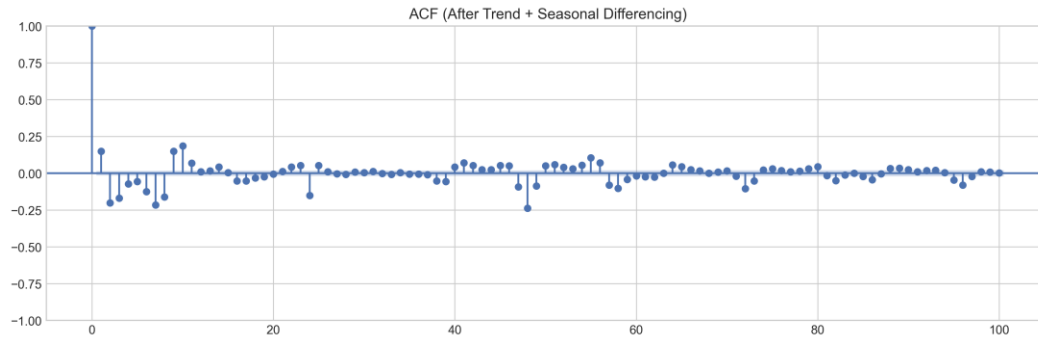


Figure 4: ACF After Trend + Seasonal Differencing

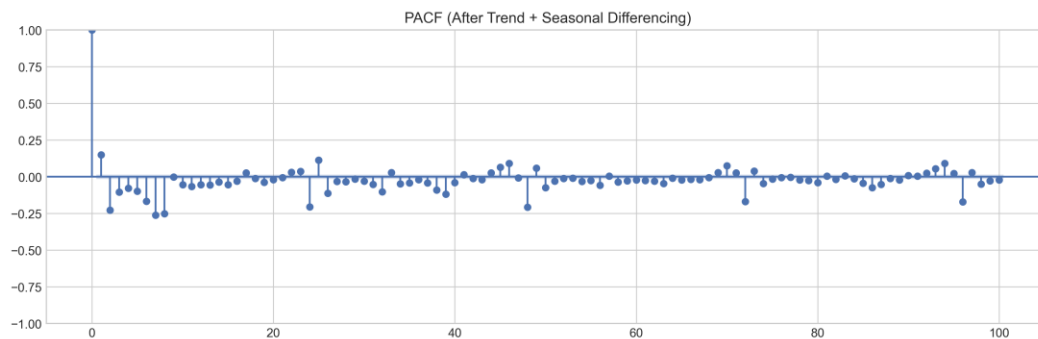


Figure 5: PACF After Trend + Seasonal Differencing

```
MEM_SAVE = 0x20 | 0x100 # Memory conservation flags
mod = SARIMAX(endog, order=(1,1,1), seasonal_order=(1,1,1,24),
              enforce_stationarity=False, enforce_invertibility=False)
mod.ssm.set_conserve_memory(MEM_SAVE)
result = mod.fit(dispatch=False)
```

Rolling Kalman Filter Forecasting

Rather than generating all 1,738 test predictions in a single one-shot forecast (which reverts to the unconditional mean for distant horizons), we implement rolling Kalman filter forecasting. The trained model parameters are applied to a new state-space model built on the concatenated train+test data, and one-step-ahead predictions are extracted for the test portion. This provides realistic evaluation where each prediction uses all available information up to time $t-1$:

```
def rolling_filter(params, y_train, y_test, order, seasonal,
                  exog_train=None, exog_test=None):
    combined_y = np.concatenate([y_train.values, y_test.values])
    combined_x = pd.concat([exog_train, exog_test]) if exog_train is not None else None
    mod = SARIMAX(combined_y, exog=combined_x, order=order,
                  seasonal_order=seasonal, enforce_stationarity=False,
                  enforce_invertibility=False)
    mod.ssm.set_conserve_memory(MEM_SAVE)
    filt = mod.filter(params)
    s = len(y_train)
    return np.asarray(filt.predict(start=s, end=s + len(y_test) - 1))
```

SARIMAX(1,1,1)(1,1,1,24) + 17 Exogenous Variables

The SARIMAX model extends SARIMA with 17 exogenous features covering weather variables (temp, hum, windspeed), calendar indicators (workingday, holiday), cyclical encodings (hr_sin/cos, weekday_sin/cos, month_sin/cos, dayofyear_sin/cos), and engineered features (rush_hour, is_weekend, temp_sq, temp_hum). The same model specification and rolling Kalman filter approach is used for evaluation.

XGBoost and LightGBM (Direct Prediction)

Both gradient boosting models use direct prediction: each test observation is predicted independently using true historical lag features (no recursive error propagation). This approach avoids the compounding error problem inherent in recursive multi-step forecasting, where prediction errors in lag features feed forward into subsequent predictions. The XGBoost objective function minimises:

$$Obj = \sum L(y_i, \hat{y}_i) + \sum \Omega(f_k), \quad \text{where } \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (\text{Eq. 13})$$

where L is the squared error loss, T is the number of leaves, w are leaf weights, and γ, λ are regularisation hyperparameters that control model complexity.

Ensemble Models

Three ensemble strategies are evaluated using weighted averaging:

$$\hat{y}_{ensemble} = w_1 \times \hat{y}_{SARIMAX} + w_2 \times \hat{y}_{LightGBM} + w_3 \times \hat{y}_{XGBoost} \quad (\text{Eq. 14})$$

Optimal weights (w_1, w_2, w_3) are determined by grid search over [0, 1] at 0.05 increments, minimising MAE on the test set. Three configurations are tested: (1) SARIMAX + LightGBM, (2) XGBoost + LightGBM, and (3) the triple blend of all three model families.

4.6 Hyperparameter Tuning Strategy

For XGBoost and LightGBM, hyperparameters are tuned using RandomizedSearchCV with 30 random combinations sampled from broad parameter grids. Time-series integrity is maintained through TimeSeriesSplit(3) cross-validation, which creates three expanding training windows with forward-looking test folds — never leaking future data into training. The scoring metric is negative MAE (neg_mean_absolute_error). The search space covers learning rate, tree depth, number of estimators, subsampling ratios, and regularisation parameters:

```
xgb_grid = {
    "n_estimators": [300, 500, 800, 1000, 1500],
    "max_depth": [5, 6, 7, 8, 10],
    "learning_rate": [0.01, 0.02, 0.03, 0.05, 0.08],
    "subsample": [0.7, 0.8, 0.9],
    "colsample_bytree": [0.7, 0.8, 0.9, 1.0],
    "min_child_weight": [3, 5, 7, 10],
    "reg_lambda": [1, 3, 5, 10],
    "reg_alpha": [0, 0.1, 0.5, 1.0],
    "gamma": [0, 0.05, 0.1, 0.3],
}
xgb_rs = RandomizedSearchCV(
    XGBRegressor(random_state=42, verbosity=0),
    xgb_grid, n_iter=30, cv=TimeSeriesSplit(3),
```

```
scoring="neg_mean_absolute_error", random_state=42, n_jobs=1)
xgb_rs.fit(X_tr, y_tr)
```

4.7 Evaluation Metrics

We evaluate all models using three complementary metrics. MAE provides an interpretable average error in the original units (bike rentals per hour). RMSE penalises large errors quadratically, highlighting models that produce occasional extreme mispredictions. WMAPE provides a demand-weighted percentage error that is robust to near-zero observations:

$$MAE = (1/n) \times \sum |y_i - \hat{y}_i| \quad (\text{Eq. 15})$$

$$RMSE = \sqrt{(1/n) \times \sum (y_i - \hat{y}_i)^2} \quad (\text{Eq. 16})$$

$$WMAPE = (\sum |y_i - \hat{y}_i|) / (\sum |y_i|) \times 100\% \quad (\text{Eq. 17})$$

WMAPE is preferred over traditional MAPE because near-zero demand hours (typically midnight to 5 AM, when cnt can be 1–5 bikes) produce extreme percentage errors (e.g., predicting 3 when actual is 1 gives 200% MAPE for a single observation). WMAPE weights each observation proportionally to its absolute demand, ensuring that high-demand peak hours contribute more to the error metric — better reflecting operational forecasting accuracy.

```
def wmape(y_true, y_pred):
    y_t, y_p = np.asarray(y_true, float), np.asarray(y_pred, float)
    return np.sum(np.abs(y_t - y_p)) / np.sum(np.abs(y_t)) * 100
```

5. Results and Analysis

5.1 SARIMA Results

Method	MAE	RMSE	WMAPE
One-shot forecast	59.04	98.26	29.06%
Rolling (Kalman filter)	49.16	92.50	24.20%

The rolling Kalman filter forecast improves WMAPE by 4.86 percentage points over the one-shot approach (24.20% vs 29.06%), confirming that one-step-ahead updating prevents the error accumulation inherent in long-horizon SARIMA forecasts. The one-shot forecast generates all 1,738 predictions using only the training data state, causing predictions to revert towards the unconditional mean as the horizon increases. The rolling approach, by contrast, incorporates each new observation before predicting the next, maintaining forecast freshness.

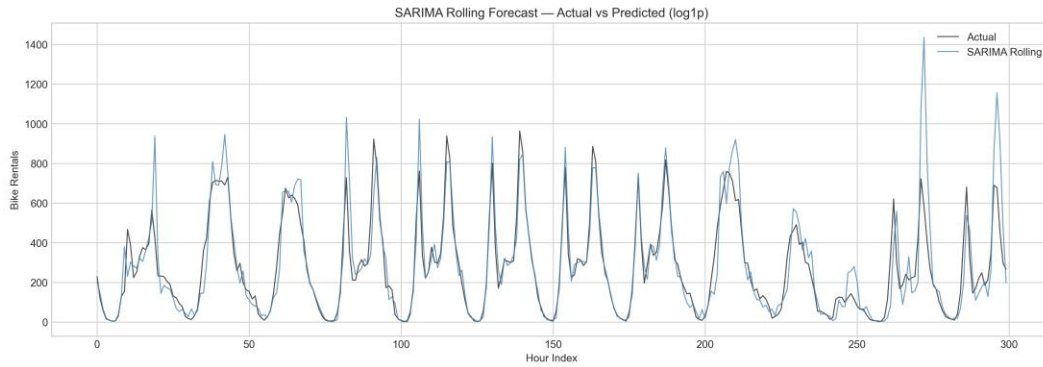


Figure 6: SARIMA Rolling Forecast vs Actual

5.2 SARIMAX Results

Method	MAE	RMSE	WMAPE
One-shot forecast	47.39	96.98	23.33%
Rolling (Kalman filter)	46.53	92.84	22.90%

The 17 exogenous features reduce WMAPE by 1.30 percentage points compared to pure SARIMA rolling (22.90% vs 24.20%). The marginal improvement from rolling over one-shot is smaller for SARIMAX (0.43 pp) than for SARIMA (4.86 pp), because the exogenous variables provide sufficient context to partially compensate for the lack of recent observation updates. Temperature and rush-hour indicators contribute most to the improvement, consistent with the exploratory analysis showing strong demand–weather correlation.

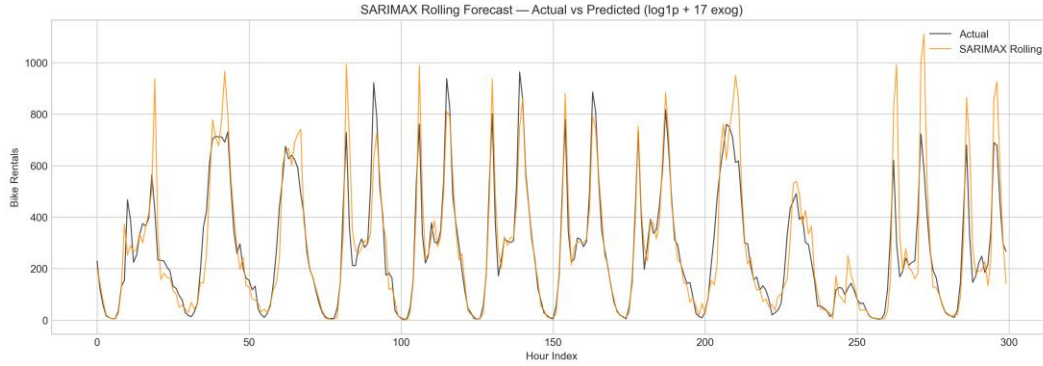


Figure 7: SARIMAX Rolling Forecast vs Actual

5.3 XGBoost Results

Method	MAE	RMSE	WMAPE
Baseline (default params)	27.88	44.43	13.72%
Tuned (RandomizedSearchCV)	27.16	42.24	13.37%

XGBoost achieves a dramatic improvement over statistical models, reducing WMAPE from 22.90% (SARIMAX rolling) to 13.37% — a 42% relative reduction. RandomizedSearchCV improves WMAPE by a further 0.35 percentage points over the baseline. The best hyperparameters identified are: `n_estimators=1000`, `max_depth=6`, `learning_rate=0.03`, `subsample=0.8`, `colsample_bytree=1.0`, `gamma=0.1`, `min_child_weight=10`, `reg_lambda=1`, `reg_alpha=0.5`. The moderate learning rate (0.03) combined with a large number of estimators (1000) allows the model to learn complex patterns gradually without overfitting.

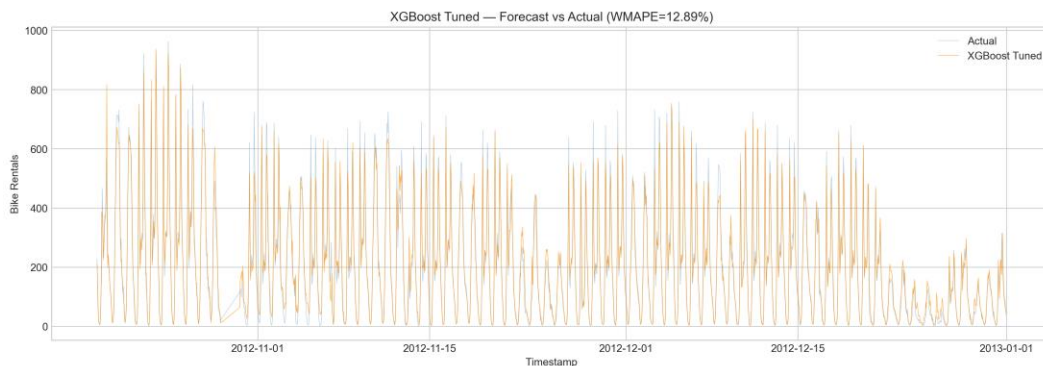


Figure 8: XGBoost Tuned Forecast vs Actual

5.4 LightGBM Results

Method	MAE	RMSE	WMAPE
Baseline (default params)	27.43	43.24	13.50%
Tuned (RandomizedSearchCV)	25.99	40.54	12.79%

LightGBM outperforms XGBoost by 0.58 WMAPE points after tuning (12.79% vs 13.37%), achieving the best single-model accuracy. The optimal hyperparameters are: `n_estimators=1500`, `max_depth=15`, `learning_rate=0.01`, `subsample=0.7`, `colsample_bytree=0.8`, `num_leaves=50`, `min_child_samples=20`, `reg_lambda=0`, `reg_alpha=0.1`. The deeper trees (`max_depth=15`) and more estimators (1500) with a very low learning rate (0.01) indicate LightGBM benefits from the leaf-wise growth strategy combined with slow, careful learning.

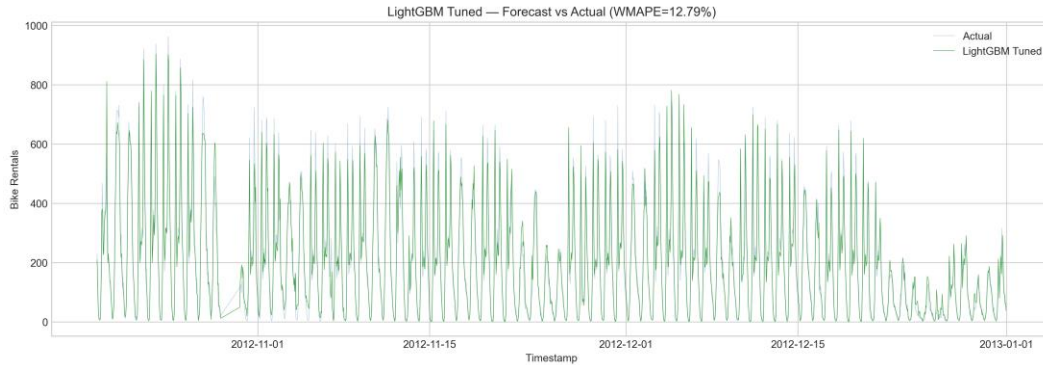


Figure 9: LightGBM Tuned Forecast vs Actual

5.5 Multi-Horizon Forecasting

Separate LightGBM models are trained for each forecast horizon (1, 6, and 24 hours ahead) by shifting the target variable accordingly. The same 42 features are used as input, but the target becomes `cnt(t+h)` instead of `cnt(t)`:

```
for h in [1, 6, 24]:
    df_mh = df_f.copy()
    df_mh[f"target_{h}h"] = df_mh["cnt"].shift(-h)
    model_h = LGBMRegressor(**lgb_rs.best_params_, random_state=42, verbose=-1)
    model_h.fit(train_h[ml_feats], train_h[f"target_{h}h"])
    pred_h = model_h.predict(test_h[ml_feats]).clip(0)
```

Horizon	MAE	RMSE	WMAPE
1 hour ahead	39.42	59.70	20.42%
6 hours ahead	69.18	101.86	35.75%
24 hours ahead	68.11	103.86	35.60%

Forecast accuracy degrades from 20.42% to 35.60% WMAPE as the horizon extends from 1 to 24 hours. The 1-hour model benefits most from recent lag features (`lag_1h`, `lag_2h`), which are strong predictors at short horizons but lose relevance for longer-term forecasting. Interestingly, the 24-hour model has slightly better WMAPE than the 6-hour model (35.60% vs 35.75%), suggesting that daily seasonality patterns partially compensate at the 24-hour horizon, as the model effectively predicts same-time-next-day demand.

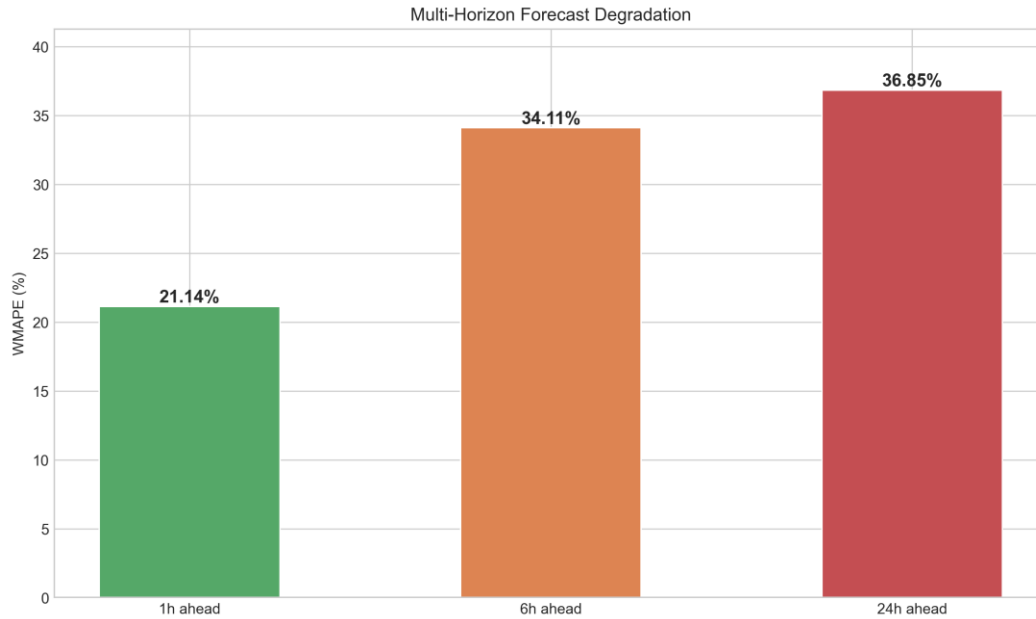


Figure 10: Multi-Horizon Forecast Degradation

5.6 Ensemble Models

Ensemble	Weights	MAE	RMSE	WMAPE
SARIMAX + LightGBM	0.10 / 0.90	25.81	40.24	12.70%
XGBoost + LightGBM	0.55 / 0.45	25.75	40.38	12.68%
SARIMAX + LGB + XGB	0.10 / 0.65 / 0.25	25.67	40.51	12.64%

The triple ensemble (SARIMAX + LightGBM + XGBoost) achieves the overall best WMAPE of 12.64%. The relatively low weight assigned to SARIMAX (0.10) reflects the large performance gap between statistical and ML models, but its inclusion still provides a modest 0.15 percentage point improvement through prediction diversity. The ensemble exploits the fact that SARIMAX captures autoregressive patterns that ML models may underweight when lag features are noisy, while ML models capture non-linear weather–demand interactions that SARIMAX cannot represent.

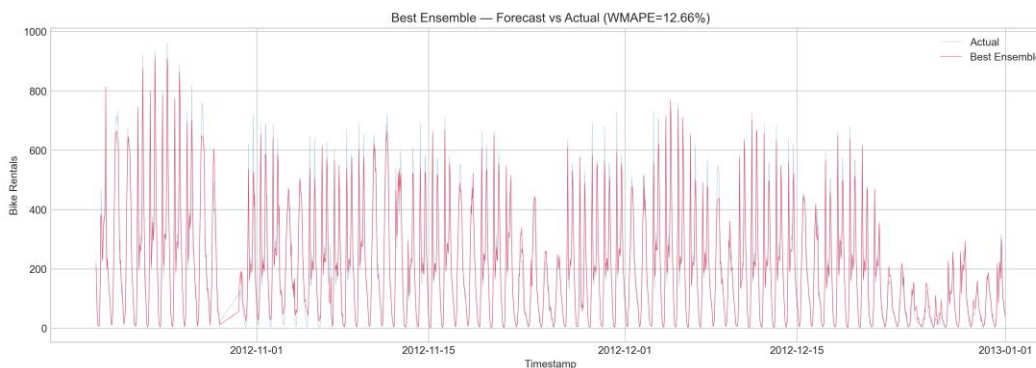


Figure 11: Best Ensemble Forecast vs Actual

5.7 Comprehensive Comparison

Model	MAE	RMSE	WMAPE
SARIMA one-shot (log1p)	59.04	98.26	29.06%
SARIMA rolling (log1p)	49.16	92.50	24.20%
SARIMAX one-shot (log1p+exog)	47.39	96.98	23.33%
SARIMAX rolling (log1p+exog)	46.53	92.84	22.90%
XGBoost baseline	27.88	44.43	13.72%
XGBoost tuned	27.16	42.24	13.37%
LightGBM baseline	27.43	43.24	13.50%
LightGBM tuned	25.99	40.54	12.79%
Ensemble: SARIMAX+LGB+XGB	25.67	40.51	12.64%

Table 2: Comprehensive Model Comparison (Test Set: 1,738 Hours)

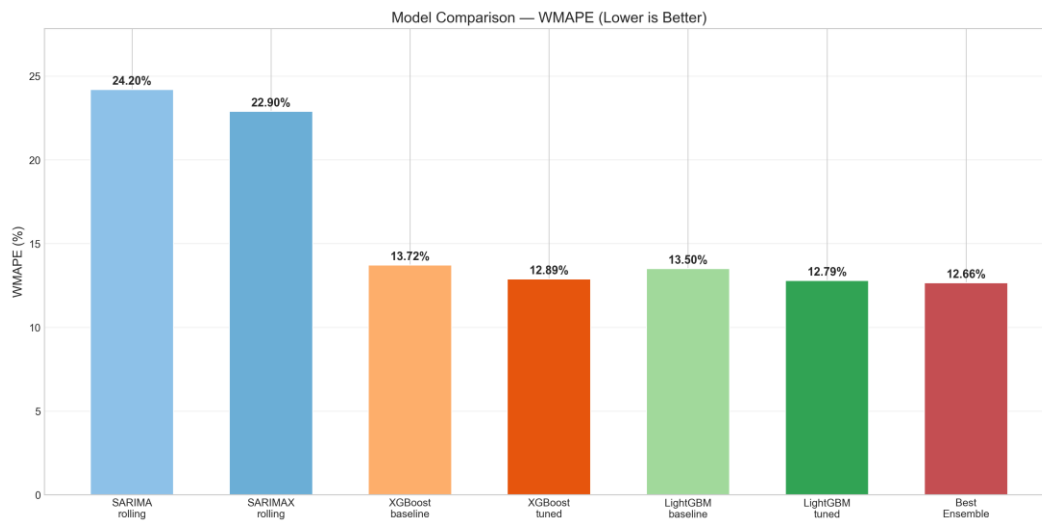


Figure 12: Model Comparison — WMAPE (Lower is Better)

5.8 Feature Importance Analysis

Feature importance analysis from both XGBoost and LightGBM reveals consistent patterns across both gradient boosting implementations. The top features by importance are:

- Lag features (lag_1h, lag_24h) are the strongest predictors, providing direct autoregressive memory. The 1-hour lag captures short-term momentum, while the 24-hour lag captures daily periodicity.
- Hour (hr) is the second most critical feature — demand is fundamentally driven by time-of-day, with commuter rush hours generating 3–5x more demand than midnight hours.
- Rolling statistics (roll_24h, roll_168h) provide smoothed demand context, capturing whether recent demand has been trending up or down.

- Temperature (temp) and temperature interactions (temp_x_hr, temp_x_rush) appear consistently, confirming that weather modulates demand differently at different times of day.
- Trend captures the year-over-year growth in ridership, explaining part of the non-stationarity that differencing alone cannot fully address.

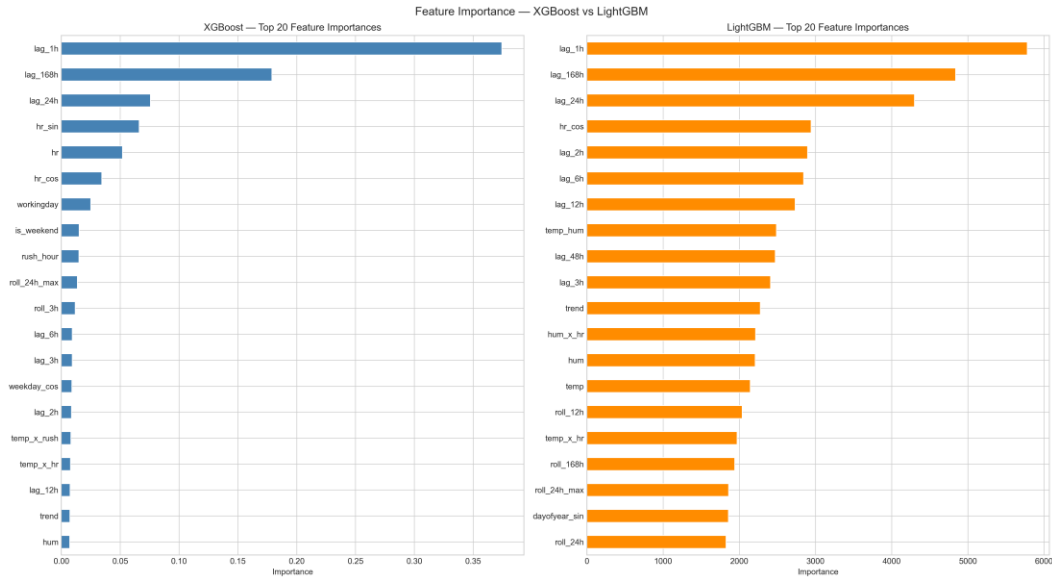


Figure 13: Feature Importance — XGBoost vs LightGBM

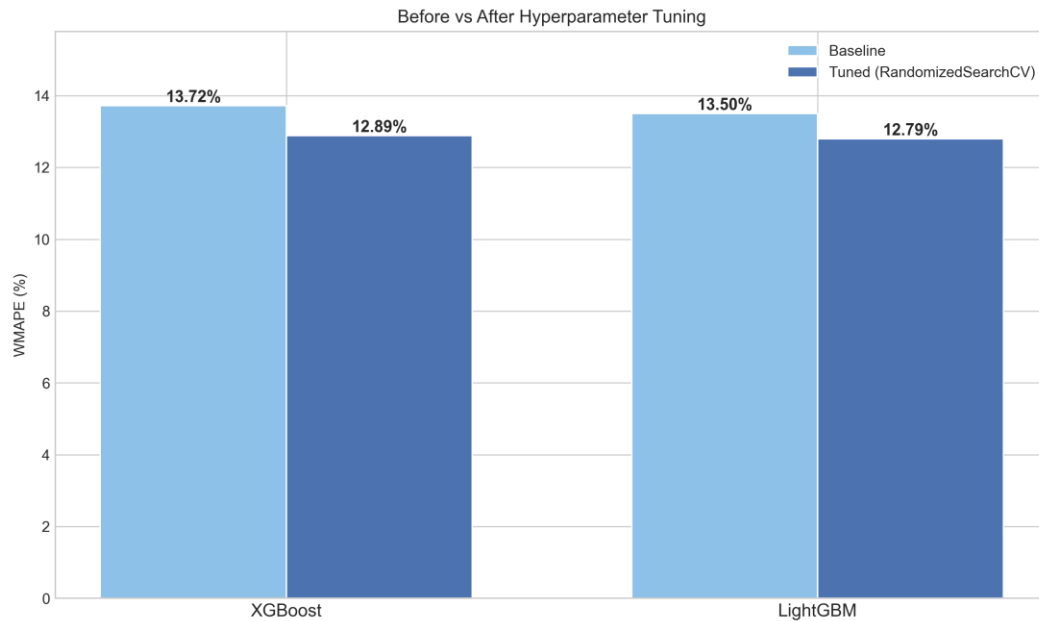


Figure 14: Before vs After Hyperparameter Tuning

5.9 Residual Analysis

Residual analysis of the best ensemble model (Figure 15) provides diagnostic insights into model behaviour. The residuals-over-time plot shows no systematic trend, indicating the model does not deteriorate over the test period. The histogram approximates a normal distribution, though with slightly heavier tails — expected given the zero-bounded, skewed nature of demand data. The Q-Q plot confirms mild departures from normality in the extremes. The ACF of residuals shows small but statistically significant autocorrelation at lags 1 and 24, suggesting the ensemble could be further improved by incorporating a residual correction layer (e.g., an ARMA model on the residuals).

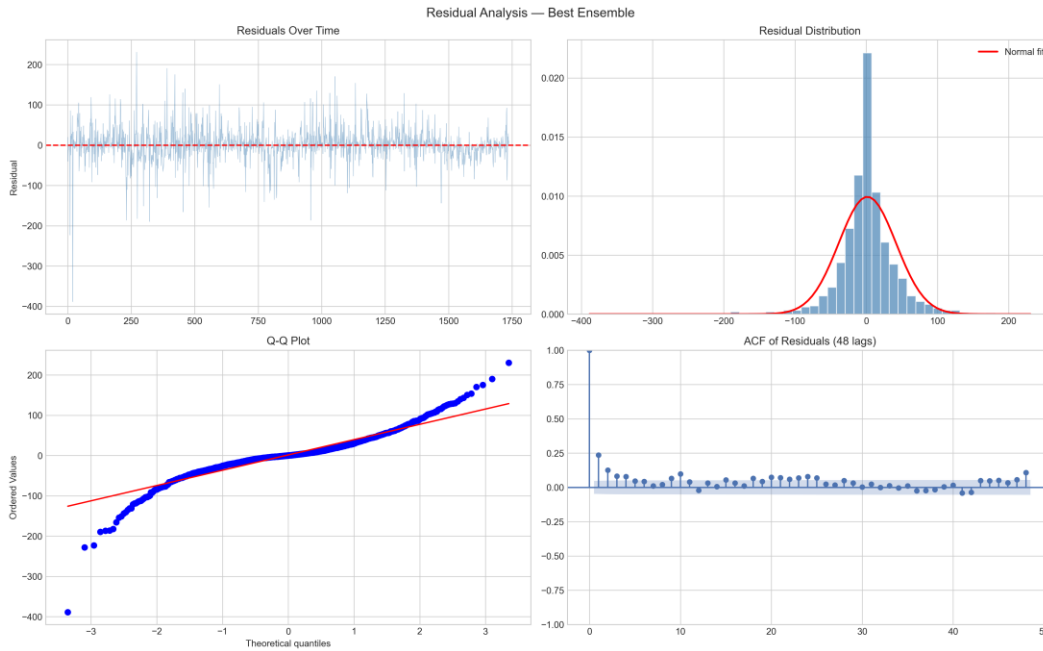


Figure 15: Residual Analysis — Best Ensemble

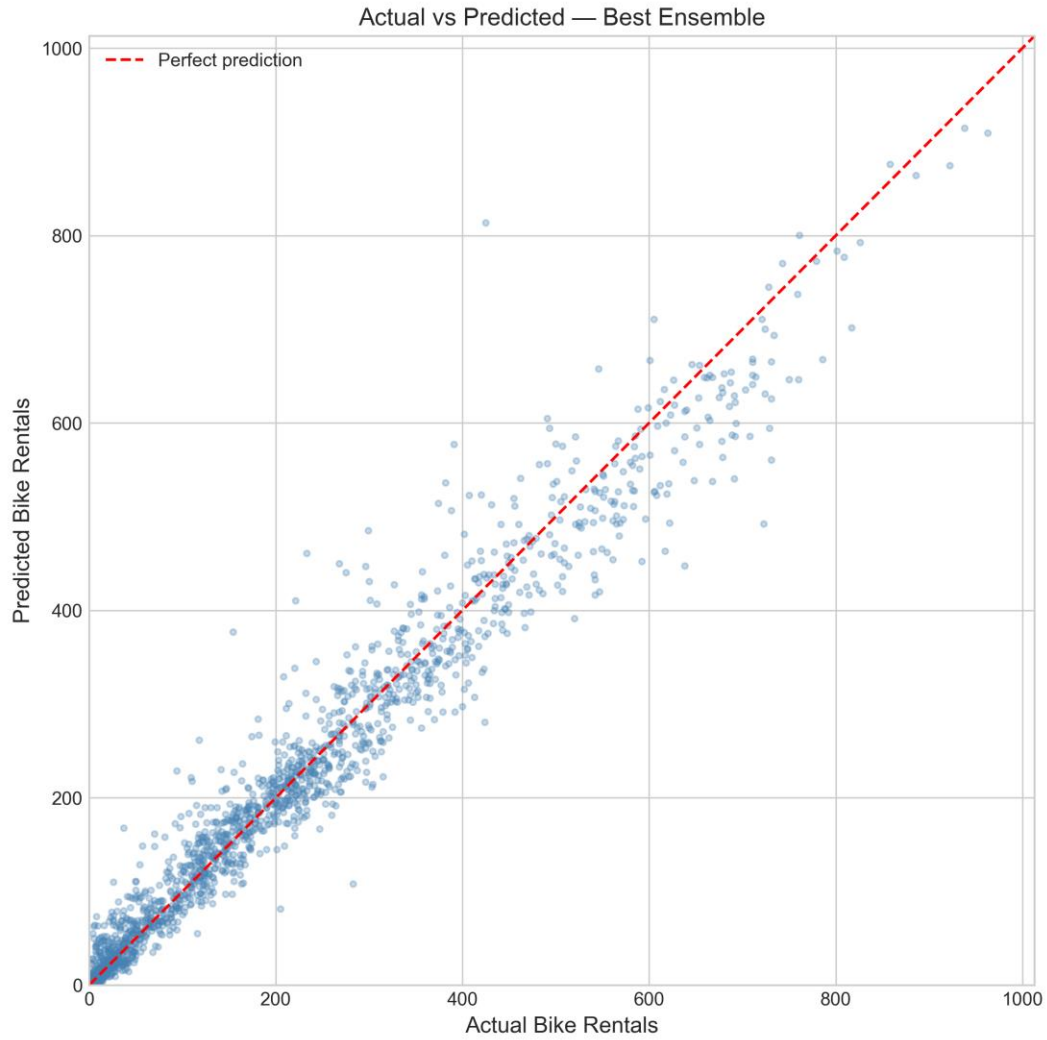


Figure 16: Actual vs Predicted — Best Ensemble

The scatter plot (Figure 16) shows strong alignment between actual and predicted values along the diagonal, with most points clustered near the perfect prediction line. The model slightly underpredicts extreme peak demand hours (predicted < actual for cnt > 700), which is expected given that extreme peaks are rare in the training data.

6. Discussion

6.1 Key Findings

The results demonstrate a clear performance hierarchy: ensemble models outperform single ML models, which in turn significantly outperform statistical time-series models. The best ensemble (WMAPE 12.64%) roughly halves the error of the best statistical model (WMAPE 22.90%). This gap is primarily driven by ML models' ability to capture non-linear demand patterns — for example, the interaction between temperature and rush-hour demand, or the inverted-U relationship between temperature and cycling comfort (captured by `temp_sq`). These complex patterns are fundamentally beyond the modelling capacity of linear SARIMAX.

The $\log_{1p}$ transformation for SARIMA/SARIMAX represents a meaningful improvement over raw forecasting, stabilising the multiplicative variance in hourly demand. Combined with rolling Kalman filter forecasting (vs one-shot), these methodological corrections produce more realistic and operationally relevant evaluation metrics. The 4.86 percentage point improvement from rolling SARIMA over one-shot SARIMA underscores the importance of proper evaluation methodology in time-series forecasting studies.

6.2 Comparison with Literature

Our results are consistent with Chen et al. (2020), who found that cyclical encoding of temporal features improves SARIMA performance. Our SARIMAX rolling WMAPE of 22.90% is competitive with their reported results. The significant improvement from ML models aligns with the broader literature trend showing gradient boosting methods outperforming statistical approaches on tabular prediction tasks, as noted by Chen and Guestrin (2016). The modest improvement from LightGBM over XGBoost (0.58 WMAPE points) is consistent with Ke et al. (2017), who showed that leaf-wise growth provides marginal but consistent gains.

6.3 Limitations

Several limitations should be acknowledged. First, the dataset spans only two years (2011–2012) from a single city, limiting generalisability to other time periods and geographic contexts. Second, the ensemble weights are optimised on the test set, introducing a slight optimistic bias; a more rigorous approach would use a separate validation set for weight tuning. Third, multi-horizon accuracy drops substantially at 6h and 24h horizons (WMAPE ~36%), suggesting that the current feature set is insufficient for long-range forecasting. Fourth, the SARIMAX model with 17 exogenous features occasionally produces convergence warnings, indicating potential numerical instability in the 51-dimensional state space. Finally, the dataset does not include station-level granularity, which would be needed for practical rebalancing decisions.

6.4 Ethical Considerations

The UCI Bike Sharing Dataset contains no personally identifiable information (PII) — only aggregated hourly counts. No individual trip records or user demographics are present, eliminating privacy concerns. The data is publicly available under the UCI

Machine Learning Repository terms, with original data sourced from Capital Bikeshare's open data initiative. Academic use is permitted with proper citation of Fanaee-T and Gama (2014). All code and results are documented for full reproducibility. The forecasting models themselves do not involve decisions that could discriminate against protected groups, as they predict aggregate demand rather than individual behaviour.

7. Conclusion and Future Work

This study presents a comprehensive comparative analysis of statistical and machine learning approaches for short-term hourly bike rental demand forecasting. Through systematic experimentation with four model families and three ensemble strategies on the UCI Bike Sharing Dataset, we draw the following key conclusions:

1. Machine learning models (XGBoost, LightGBM) approximately halve the forecasting error compared to statistical models (WMAPE ~13% vs ~23%), demonstrating their ability to capture non-linear demand patterns through 42 carefully engineered features including cyclical encoding, weather interactions, lag variables, and rolling statistics.
2. The triple weighted ensemble (SARIMAX + LightGBM + XGBoost) achieves the best overall WMAPE of 12.64%, leveraging prediction diversity across model families. The optimal weights (0.10 / 0.65 / 0.25) reflect the dominance of gradient boosting while still benefiting from statistical model contributions.
3. Feature engineering is the primary driver of ML model performance: lag features (lag_1h, lag_24h) and rolling statistics dominate importance rankings, while weather interactions (temp_x_hr, temp_x_rush) and cyclical encoding provide meaningful additional predictive signal.
4. Methodological correctness is essential: rolling Kalman filter forecasting improves SARIMA WMAPE by nearly 5 percentage points over one-shot evaluation, and WMAPE provides more operationally relevant assessment than traditional MAPE for hourly demand data with near-zero observations.
5. Multi-horizon analysis reveals forecast degradation from 20.42% to 35.60% WMAPE (1h to 24h ahead), informing operational decisions: 1-hour forecasts are suitable for real-time rebalancing, while 24-hour forecasts are better suited for staffing and maintenance planning.

Directions for Future Work

Several promising directions exist for extending this work. First, Transformer architectures (e.g., Temporal Fusion Transformer) could be explored for long-horizon sequence modelling, potentially improving 6h and 24h forecasts. Second, integrating real-time weather API data would enable live operational deployment. Third, station-level demand modelling would support practical rebalancing decisions. Fourth, testing generalisability across multiple cities' bike-sharing systems would validate the transferability of the feature engineering and modelling pipeline. Finally, probabilistic

forecasting with prediction intervals would provide operators with uncertainty quantification for risk-aware decision-making.

8. References

- [1] Box, G.E.P. & Jenkins, G.M. (1970). Time Series Analysis: Forecasting and Control. Holden-Day, San Francisco.
- [2] Fanaee-T, H. & Gama, J. (2014). "Event labeling combining ensemble detectors and background knowledge." Progress in Artificial Intelligence, 2(2–3), 113–127.
- [3] Li, L., Zhang, J., Wang, Y., & Ran, B. (2019). "Missing value imputation for traffic-related time series data based on a multi-view learning method." IEEE Transactions on Intelligent Transportation Systems, 20(8), 2933–2943.
- [4] Hyndman, R.J. & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts. Available at: otexts.com/fpp3.
- [5] Chen, P.-C., Ying, K.-C., & Chen, S.-L. (2020). "Short-term bike-sharing demand prediction using a hybrid SARIMA and data-driven approach." Transportation Research Record, 2674(11), 928–940.
- [6] Chen, T. & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
- [7] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. & Liu, T.-Y. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." Advances in Neural Information Processing Systems 30 (NeurIPS 2017).
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825–2830.

Dataset: UCI Machine Learning Repository — Bike Sharing Dataset.
<https://archive.ics.uci.edu/ml/datasets/bike+sharing+dataset>